Choosing Versus Accepting: Why Better Coding Assistants Can Leave More Technical Debt

Ook Lee, *Hanyang University, Seoul, 04763, South Korea*

Abstract*—Large language model (LLM) coding assistants change the order of a developer's decision. Without an assistant, the developer decides how to build a task before any code exists; with one, working code arrives first and the decision becomes whether to accept it. Drawing on a formal model of this reordering, we show that rising technical debt alone cannot reveal whether developers decide differently, because a rational response to faster work produces the same pattern. Three measures can: how much of the debt developers flag, how much more often they defer refactoring when working code is already in hand, and how often planned refactoring never happens. The model also implies that more reliable models can leave more debt, that codebases can tip into a high-debt state, and that a temporary review mandate can have lasting effects. We report commit data consistent with these predictions.

Actionable insights for practitioners

- *Do not judge an assistant by total debt*—Track how much of the debt developers flag, whether planned refactoring actually happens, and how much review a model upgrade replaces.
- *Decide before generating*—Asking developers to commit to a quick or a careful approach before the assistant produces code removes the order effect described here.
- *Use review mandates to correct beliefs*—A review mandate that lasts long enough to correct developers' sense of how often generated code is wrong can keep working after it ends; a shorter one does not.

The standard economic account of technical debt treats each piece of work as a trade-off. A shortcut saves time today and costs rework later, and a team takes it when the time saved is worth more than the rework it will require.[1] On this account, debt should fall when quality becomes cheaper, and LLM coding assistants have made quality much cheaper. The first evidence points the other way. In open-source projects that adopted an agentic assistant, velocity rose only briefly while static-analysis warnings and code complexity rose persistently,[2] and an audit of code generated by LLMs and coding agents found that more capable models produce more bloated and tightly coupled code.[3]

This article argues that assistants change the structure of the quality decision more than its

XXXX-XXXX © 2021
Digital Object Identifier 10.1109/XXX.0000.000000

price. Without an assistant, a developer chooses an approach before either implementation exists. With one, working code arrives first, often a quick solution that looks finished, and the decision becomes whether to accept it or to spend further effort improving it. We call this the shift from choosing to accepting. We summarize a formal model of the shift, explain what it implies for how teams should evaluate and oversee assistants, and report repository data that bear on its predictions.

RELATION TO PRIOR WORK

A growing literature documents what happens to generated code. Liu and colleagues traced about 303,000 AI-authored commits across 6,299 repositories and found that 22.7% of the issues those commits introduced were still present in the latest revision.[4] A multivocal review concludes that LLM assistance amplifies code, design, and documentation debt.[5] Among 6,540 code comments that mention LLMs, only about 1.2% also admit technical debt.[6] The role of judgment has been examined as well. Interviews with architects have linked anchoring and optimism to architectural debt,[7] and an observational study of developers working with LLMs classified 48.8% of their actions as biased.[8]

These studies establish that generated code carries persistent issues and that developers rarely record them. They do not explain why, and they cannot say whether the pattern is simply a rational response to cheaper code. Our contribution is to separate the two explanations and to identify what would distinguish them in data.

FROM CHOOSING TO ACCEPTING

The model compares two workflows that differ only in when the developer decides. In the decide-first workflow, the developer commits to a quick or a careful approach before doing the work. Both the extra effort and the rework it avoids then lie in the future, so the decision depends only on their relative size. In the generate-first workflow, working code arrives first. The developer may notice that it is flawed, may spend time reviewing it, and, if a flaw is found, may refactor it, while the rework from an unrepaired flaw comes later.

Three well-documented biases enter the model. The first is the pull of the present: developers give effort spent now more weight than effort saved later.[9] The second is misplaced trust: code that looks finished is judged less likely to be wrong than it is, as research on automation bias would predict.[10] The third is reluctance to change what already works: rewriting code that is already in hand feels more costly than building it differently would have been.[11] A fourth factor, introduced later, is a share of suggestions that are accepted by default without any real decision. The sidebar "The Model in Brief" states these assumptions formally.

The central observation is that these biases have nothing to act on in the decide-first workflow. When the decision comes before both the effort and the code, the pull of the present weighs both sides equally, there is no output to trust too much, and nothing is yet in hand to protect. Each bias moves the decision once working code exists. The order of events, rather than the presence of bias, therefore determines whether the biases matter.

THE MODEL IN BRIEF

A task can be built quickly (E) or carefully (Q). Building Q takes a more units of time, valued at v per unit; a task left as E costs rework R later, discounted by δ.

Decide first. The developer chooses E if $va > \delta R$. Every cost lies in the future, so the biases below cancel out.

Generate first. Generated code is flawed with probability q. A flaw that is found is refactored, at time cost r, only if $\lambda vr < \beta\delta R$. Code not seen to be flawed is reviewed only if the expected gain exceeds the review time.

Biases. $\beta \leq 1$, the pull of the present (present bias); $\varphi \leq 1$, the share of the true error rate that developers perceive (misplaced trust); $\lambda \geq 1$, the extra weight on changing code already in hand (loss aversion); α, the share of suggestions accepted without review. A fully rational developer has $\beta = \varphi = \lambda = 1$ and $\alpha = 0$.

What the model shows. With all biases off, the flagged share, the working-code effect, and broken plans take their rational values whatever the error rate or pace of work. Each bias moves exactly one or two of them, as Table 1 summarizes.

Numerical results quoted in the text use illustrative values of these parameters, not estimates.

WHY RISING DEBT PROVES LITTLE

Rising debt might seem to confirm the argument, but a fully rational developer can also accumulate more debt when code becomes cheaper. When a team can build faster, shipping sooner becomes more valuable, and quick solutions become more attractive. We call this the speed-up explanation. The model splits the change in debt after adoption into three parts: cheaper quality lowers debt, faster work raises it, and the biases raise it further.

All numbers in this and the following section come from the model with parameter values that we chose to illustrate the mechanisms. They are not estimates, and they show what the mechanisms can produce, not how large they are in practice. In one such illustration, cheaper quality lowers the probability that a task ends with debt by 0.24 and the biases raise it by 0.29, so debt rises from 0.500 to 0.555 per task even though quality has become cheaper.

Crucially, whatever the biases do to the level of debt, some faster pace of work leads a fully rational developer to exactly the same level. Aggregate data, including the difference-in-differences estimates that dominate current evidence, therefore cannot tell the two explanations apart. Three measures can, because each stays at a fixed value under the speed-up explanation and moves in a known direction once a bias applies (see Table 1):

- *Flagged share of debt*—the share of debt in the code that developers have marked, for example with TODO or FIXME comments.[12] It falls when developers trust generated code too much.
- *Working-code effect*—how much more often developers put off refactoring when working code is already in hand than when they make the same choice in advance. It is zero for a fully rational developer and positive under the pull of the present or reluctance to change working code.
- *Broken refactoring plans*—the share of planned refactoring that never happens. Only the pull of the present produces it, so it separates that bias from reluctance to change working code.

TABLE 1. Predictions that separate the speed-up explanation from the behavioral account.

Prediction	Speed-up explanation	Behavioral account
Total debt after adoption	Rises	Rises
Flagged share of debt	Unchanged, no difference by authorship	Lower for generated code
Working-code effect on refactoring	Zero	Positive
Broken refactoring plans	None	Present
The three measures when the decision precedes generation	Unchanged	Return to rational values
Debt as models become more reliable	Rises over a narrow range	Rises over a wider, shifted range
Effect of a temporary review mandate after it ends	Ends with the mandate	Can persist

In the illustration above, the speed-up explanation reproduces the same level of debt if the value of developer time rises by a factor of about 2.6. Even then, the flagged share stays at its baseline of 0.50, and there is no working-code effect and no broken plans. With the biases, the flagged share falls to 0.33, the working-code effect is 0.31, and 0.21 of tasks end with a broken refactoring plan. A rational developer may also change course because the generated code reveals something about the work ahead. The model limits how large that effect can be, so a working-code effect beyond that limit cannot be explained by new information alone.

THREE RESULTS THAT MATTER FOR PRACTICE

Better Models Can Leave More Debt

When generated code is often wrong, developers review nearly every suggestion and little debt gets through. As the model improves, developers sense less risk and stop reviewing before the error rate has fallen far enough to compensate. Over a range of error rates, the share of flaws that reach the codebase then rises faster than the error rate falls. The pattern is familiar from human factors research. A meta-analysis of 18 experiments found that higher degrees of automation improve routine performance but worsen performance when the automation fails,[13] and in a field experiment with 181 recruiters, those given higher-quality AI recommendations spent less effort on each candidate.[14] What the model adds is the consequence for accumulated debt: a fully rational developer also reviews less as models improve, but the biases widen the range of error rates over which better models leave more debt and move it toward the error rates of current models. In the illustration in Figure 1, which uses assumed rather than measured values, debt per task rises by 56% as the error rate falls from 0.46 to 0.29. The direction of the effect, not its size, is the result: the same upgrade can reduce debt in one organization and raise it in another.

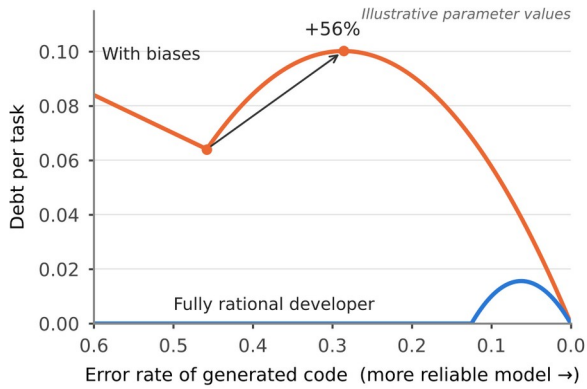


FIGURE 1. Better models can leave more debt. Debt per task as the error rate of the model falls, for a fully rational developer and for one subject to the biases in the sidebar. The curves are computed from the model with illustrative parameter values (present bias 0.5, perceived share of errors 0.6, loss aversion 1.2, default acceptance 0.2), not estimated from data; they show the shape of the effect, not its size.

Codebases Can Tip Into a High-Debt State

Debt accumulates, and the more of it a codebase carries, the more each refactoring costs, which is the interest in the metaphor. Under the pull of the present, a codebase can settle in either of two stable states, one with little debt and one with much more. In an illustrative parameterization, the high-debt state carries ten times the debt of the low one. Because the pull of the present does not act in the decide-first workflow, adopting an assistant can itself tip a codebase into the high-debt state even though the assistant also makes refactoring cheaper. Once there, the codebase does not return when conditions improve slightly. Small nudges that would have prevented the tipping cannot reverse it; escape requires a large change in how decisions are made or a one-time refactoring program.

Temporary Mandates Can Have Lasting Effects

Developers can learn how often generated code is wrong, but mainly by reviewing it, which is exactly what misplaced trust discourages. Developers who believe generated code is reliable enough not to review never see its flaws and never revise the belief. Economists call such a belief self-confirming, because the behavior it produces withholds the evidence that would correct it. A mandate that requires review for a limited period breaks this loop if it lasts long enough for developers to see enough flaws to start reviewing on their own. In the model's illustration, with assumed values, a single period of mandatory review suffices. Under the speed-up explanation, by contrast, the effect of a mandate should end when the mandate does, which gives a direct test.

WHAT REPOSITORY DATA SHOW

Archival data cannot test the mechanisms, since aggregate outcomes do not separate the explanations and the biases are not observed, but they can be consistent or inconsistent with the predictions. We examined six actively maintained open-source projects with many contributors: Prefect, Cal.com, n8n, Grafana, Home Assistant, and Sentry. A commit was treated as AI-assisted when its message carried a co-author trailer or footer naming a coding assistant such as Claude, Codex, Cursor, or Copilot. We analyzed nonmerge commits from April 2025 to October 2026, excluding bot accounts, nonsource files, generated code, and commits that added more than 5000 lines.

To hold skill and style constant, we compared each developer with themselves. For every developer with at least five AI-labeled commits, we compared those commits with the same developer's unlabeled commits made after the first labeled one, retaining developers with at least five of each. This left 251 developers in five projects and 32,620 commits, of which 10,641 carried an AI label (see Table 2). Unlike the

figures in the previous sections, these are measured values.

TABLE 2. Within-developer comparison of AI-labeled and other commits (251 developers, five projects). Measured values.

Indicator	AI-labeled commits	Other commits	Developers for whom AI-labeled commits score higher	*p*
Debt markers (TODO, FIXME, HACK, XXX) per 1000 added lines, all commits pooled	0.16	0.20	22% (29% lower, 49% tied)	.16[b]
Additions as share of changed lines, median across developers	82.5%	78.8%	61%	.002[a]
Added lines duplicated within the same commit, median across developers	32.3%	33.0%	49%	.93[a]
Changed lines per commit, median across developers	152	85	72%	<.001[a]

[a]Wilcoxon signed-rank test across developers.

[b]Sign test excluding ties.

AI-labeled commits carried about 20% fewer debt markers per added line, which matches the direction the flagged share predicts, although the within-developer difference was not statistically reliable and fewer markers could also reflect less debt. AI-labeled commits were also more weighted toward additions for 61% of developers, which is consistent with reluctance to remove generated code. That difference narrowed and lost statistical reliability among commits adding 10 to 300 lines (57% of 170 developers, $p = .23$), because AI-labeled commits were considerably larger. Duplication within commits did not differ. Labels capture only disclosed assistance, which biases all comparisons toward zero.

A separate analysis of 431,043 agent-authored and 2,525,598 human pull requests on GitHub, drawing on event logs and the AIDev dataset,[15] points the same way. Within repositories, agent pull requests had a ratio of additions to deletions about 1.8 times higher, carried 0.62 debt markers per 1000 added lines against 1.38 for human pull requests, and in team repositories, excluding Copilot, were 5.9 percentage points less likely to be reviewed. These patterns are consistent with the theory but cannot identify its mechanisms.

IMPLICATIONS FOR TEAMS

Remedies such as debt registers and repayment sprints assume that debt is visible and that it will be repaid when the numbers justify it. Where the biases operate, less debt is recorded and planned repayment does not happen, so those remedies will underperform. The model suggests four alternatives:

- *Decide before generating*—Ask developers to commit to an approach before the assistant produces code. The three measures then return to their rational values.
- *Evaluate upgrades by the review they replace*—Measure review effort and the flagged share alongside error rates whenever a model is upgraded.
- *Use mandates to correct beliefs*—Set the length of a review mandate by whether it corrects developers' sense of how often generated code is wrong, not by how long control is needed.
- *Prevent tipping rather than reverse it*—Watch codebases that enter adoption with high debt, since they are the most likely to

tip, and budget a one-time refactoring program for those that do.

LIMITATIONS

The model is deliberately minimal, and its results describe mechanisms rather than magnitudes; the numbers quoted from it are illustrations, not estimates. Tasks are independent, review finds every flaw it looks for, and the tipping result relies on a particular form of interest on debt. The results about the pull of the present assume that, without an assistant, the quality decision is made before effort is committed. Learning concerns the error rate and not how polished the output looks, which may resist statistical evidence.

The theory is untested. The repository data are observational, drawn from public projects, and silent on the industrial settings in which most commercial software is built. Agent labels are incomplete, and static analysis only approximates the debt developers experience. The decisive test is an experiment that observes the same decision when developers choose in advance, plan ahead, and accept generated code, with information held fixed, ideally with professional developers in industry.

CONCLUSION

LLM coding assistants have made code cheap to produce, and the first evidence suggests that they have made systems more expensive to maintain. We have argued that the cause lies in how assistance restructures the decision, since working code now arrives before the quality question is posed. A rational response to faster work can produce the same rise in debt, so the behavioral account must be judged by what it alone predicts: debt that goes unflagged and refactoring plans that are not carried out. Two further warnings follow. Better models do not guarantee less debt, and a codebase that has tipped into a high-debt state will not be rescued by the nudges that would have prevented it.

ACKNOWLEDGMENTS

The author used an AI assistant (Claude, Anthropic) for language editing, for adapting the manuscript to this magazine, for drawing Figure 1 from the model, and for writing the commit-mining scripts behind Table 2. The author provided the model, its results, and the pull request analysis, reviewed all AI-assisted content, and takes full responsibility for the article.

REFERENCES

1. W. Cunningham, "The WyCash portfolio management system," in *Addendum Proc. OOPSLA*, 1992, pp. 29–30.
2. H. He, C. Miller, S. Agarwal, C. Kästner, and B. Vasilescu, "Speed at the cost of quality: How Cursor AI increases short-term velocity and long-term complexity in open-source projects," in *Proc. 23rd Int. Conf. Mining Softw. Repositories*, 2026, arXiv:2511.04427.
3. Y. Zhu, N. Tsantalis, and P. C. Rigby, "AI-generated smells: An analysis of code and architecture in LLM and agent-driven development," 2026, arXiv:2605.02741.
4. Y. Liu, R. Widyasari, Y. Zhao, I. C. Irsan, J. Chen, and D. Lo, "Debt behind the AI boom: A large-scale empirical study of AI-generated code in the wild," 2026, arXiv:2603.28592.
5. R. Ehsani, S. Rawal, Y. Cai, and P. Chatterjee, "Faster code, deeper debt? A multivocal literature review on technical debt and its early signs in LLM-assisted software development," *ACM Trans. Softw. Eng. Methodol.*, 2026, doi: 10.1145/3820165.
6. A. A. Mujahid and M. M. Imran, "'TODO: Fix the mess Gemini created': Towards understanding GenAI-induced self-admitted technical debt," in *Proc. 9th Int. Conf. Tech. Debt*, 2026, arXiv:2601.07786.
7. K. Borowa, A. Zalewski, and S. Kijas, "The influence of cognitive biases on architectural technical debt," in *Proc. IEEE 18th Int. Conf. Softw. Archit. (ICSA)*, 2021, doi: 10.1109/ICSA51549.2021.00019.
8. X. Zhou, Z. Saghi, S. Sabouri, R. Pandita, M. McGuire, and S. Chattopadhyay, "Cognitive biases in LLM-assisted software development," in *Proc. IEEE/ACM 48th Int. Conf. Softw. Eng. (ICSE)*, 2026, doi: 10.1145/3744916.3773104.
9. T. O'Donoghue and M. Rabin, "Doing it now or later," *Amer. Econ. Rev.*, vol. 89, no. 1, pp. 103–124, 1999.
10. R. Parasuraman and D. H. Manzey, "Complacency and bias in human use of automation: An attentional integration," *Hum. Factors*, vol. 52, no. 3, pp. 381–410, 2010.

11. D. Kahneman and A. Tversky, “Prospect theory: An analysis of decision under risk,” *Econometrica*, vol. 47, no. 2, pp. 263–291, 1979.
12. A. Potdar and E. Shihab, “An exploratory study on self-admitted technical debt,” in *Proc. IEEE Int. Conf. Softw. Maint. Evol. (ICSME)*, 2014, pp. 91–100.
13. L. Onnasch, C. D. Wickens, H. Li, and D. Manzey, “Human performance consequences of stages and levels of automation: An integrated meta-analysis,” *Hum. Factors*, vol. 56, no. 3, pp. 476–488, 2014, doi: 10.1177/0018720813501549.
14. F. Dell’Acqua, “Falling asleep at the wheel: Human/AI collaboration in a field experiment on HR recruiters,” working paper, 2022.
15. H. Li, H. Zhang, and A. E. Hassan, “AIDev: Studying AI coding agents on GitHub,” in *Proc. 23rd Int. Conf. Mining Softw. Repositories*, 2026, doi: 10.1145/3793302.3797249.

OOK LEE is a [position] in the [department] at Hanyang University, Seoul, 04763, South Korea. His research interests include [interest 1], [interest 2], and [interest 3]. Lee received the [degree] in [field] from [institution]. He is a member of [IEEE / IEEE Computer Society]. Contact him at ooklee@hanyang.ac.kr.